

Ödev 11: Tipli Bir Önbellek

Jenerik bir kap kurun, bilerek üç tip silme kısıtına çarpın ve yığın kirlenmesine sebep olup suçun tam olarak nereye düştüğünü görün.

Birinci Bölüm: Önbellek

`TypedCache.java` dosyasını klasik biçimde oluşturun.

Sabit kapasiteli bir `Cache<K, V>` yazın:

- `put(K key, V value)` — doluyken eklemeden önce **en eski** girdiyi çıkarsın
- `get(K key)` — değeri ya da `null` döndürsün
- `size()` ve `keys()`

`Cache<String, Integer>` ile çalıştığını gösterin, çıkarmanın gerçekleştiğini kanıtlayın, sonra tek bir uygulamanın ilgisiz tiplere hizmet ettiğini göstermek için aynı sınıfı `Cache<Integer, List<String>>` olarak kullanın.

Kısıtlara Çarpın

Yazarken şunların her birini deneyin, derlenmediğini doğrulayın, sonra başka biçimde çözün. Üçünü de sınıfa yazacağınız bir yorumda belgeleyin:

- Değerleri bir `V[]` dizisinde saklayın
- `if (value instanceof V)` ile doğrulayın
- `static V defaultValue;` ile bir yedek paylaşın

Her biri için hatanın ne olduğunu ve onun yerine ne yaptığınızı söyleyin.

İkinci Bölüm: Sınırlar ve PECS

Üç metot yazın:

Metot	İmza biçimi
<code>largest(List<T>)</code>	<code>compareTo</code> çağırabilmesi için sınır gerekiyor
<code>average(...)</code>	sayı okur, asla yazmaz
<code>drainInto(...)</code>	tam sayı yazar, asla okumaz

`average` ve `drainInto` metotlarına doğru joker karakterleri verin. `average` metodunun hem `List<Integer>` hem `List<Double>` üzerinde, `drainInto` metodunun da hem `List<Number>` hem `List<Object>` içine çalıştığını kanıtlayın.

Açıklayın

Kendi kelimelerinizle şunları kapsayan bir yorum yazın:

- `largest` neden sınırı olmadan çalışamaz
- `average` neden bir joker karakteri, `drainInto` neden diğerini kullanıyor
- Her joker karakterin neyi **feda ettiği**, ve o fedanın onu neden güvenli kıldığı

Yalnızca "producer extends, consumer super" ifadesini tekrarlayan cevaplar sayılmaz. Her durumda derleyicinin size neyi yaptırmayacağını söyleyin.

Üçüncü Bölüm: Yığını Kirletin

Bir `List<String>` kurun, **ham** bir `List` değişkenine atayın ve ham referans üzerinden bir `Integer` ekleyin.

Sonra sırayla şunları yazdırın:

1. Listenin içeriği
2. Eleman 0, `String` olarak okunmuş
3. Eleman 1, `String` olarak okunmuş, ne olduğunu yakalayarak

Asıl Önemli Soru

Şunu cevaplayan bir yorum yazın: **hangi satır kuralı bozdu ve hangi satır fırlattı?**

Sonra neden farklı satırlar olduklarını, derleyicinin nereye ne eklediğini ve bunun neden normal bir çökmeden daha zor hata ayıklandığını açıklayın.

Kabul Kriterleri

- ☐ `java TypedCache.java` ile çalışıyor
- ☐ `Cache<K, V>` her iki parametrede jenerik ve en eski girdiyi çıkarıyor
- ☐ Aynı `Cache` sınıfı iki ilgisiz tip argümanı çiftiyle kullanılıyor
- ☐ Bir yorum üç tip silme kısıtını ve kullanılan çözümü belgeliyor
- ☐ `largest` bir sınır bildiriyor, neden gerekli olduğunu söyleyen yorumla
- ☐ `average` ve `drainInto` doğru joker karakterleri kullanıyor
- ☐ İki de iki farklı eleman tipine karşı kanıtlanıyor
- ☐ Açıklama her joker karakterin neyi yasakladığını isimlendiriyor
- ☐ Yığın kirlenmesi yeniden üretiliyor ve `ClassCastException` yakalanıyor
- ☐ Bir yorum kuralı bozan satır ile fırlatan satırı farklı olarak tanımlıyor

İleri Seviye

`Cache` sınıfını, en son eklenen yerine **en az kullanılan** girdiyi çıkaracak şekilde değiştirin, böylece bir `get` çağrısı kullanım sayılsın.

`LinkedHashMap` bunu sizin için tek bir kurucu argümanı ve tek bir geçersiz kılınan metotla yapabilir. Bulun, kullanın ve aksi hâlde elle yazmanız gerekecekleri bir yorumda anlatın.

İpucu, üçüncü bölüm fırlatmıyorsa

Eleman 1'i `var` ya da `Object` içine değil bir `String` değişkene okuduğunuzdan emin olun. Dönüşüm, **okuma yerindeki bildirilen tip** yüzünden eklenir. Onu `Object` olarak okursanız dönüşüm olmaz, dolayısıyla başarısızlık da olmaz ve kirlenmiş liste biri onu düzgün okuyana kadar sessizce orada oturur.